视觉-文本大模型 + 机器学习算法 八股问答

> 与《八股知识问答 118 问》的分工：那份是**多模态总览 + 项目接口话术**，这份是**VLM 深度知识 + 机器学习算法原理推导**，两份不重复、可对照看。 编号用 A（视觉-文本大模型，45 问）、B（机器学习算法，40 问），避免与 Q1~Q118 冲突。 难度：★★★ 几乎必问 · ★★ 大概率问 · ★ 加分题。

Part A 视觉-文本大模型（VLM）

A1 模型族谱与架构（A1–A12）

A1 ★★★ VLM 有哪几种架构范式？各自的代表模型和取舍？

A：四种范式，按"视觉怎么进语言模型"分：

范式	结构	代表	取舍
① 双塔对比（encoder-only）	图像塔 + 文本塔，各自编码后算相似度	CLIP、ALIGN、SigLIP	检索/零样本分类强，**不能生成**，只学全局对齐
② 融合编码器	图文进同一个 Transformer（cross-attention 交互），输出融合表征	ALBEF、BLIP、ViLT	适合匹配/理解类任务，参数量比双塔小，但仍不生成自由文本
③ 编码器-解码器桥接（主流 VLM）	冻结/半冻结视觉塔 → 桥接层 → LLM 自回归生成	BLIP-2、LLaVA、Flamingo、Qwen-VL、InternVL	**能力最强、可指令化**，代价是 token 多、训练分阶段
④ 原生/早期融合（Early fusion）	一开始就把图像 patch 与文本 token 拼进同一个序列，统一自回归	Chameleon、Emu3、Fuyu	结构最简洁、天然支持交错图文，但需从零训或大改，工程门槛高

面试答法："主流是第 ③ 种——因为它的训练成本最低（可冻结双塔）、能力上限又最高；第 ④ 种是方向但需要从头训，数据门槛太高。"

追问：Fuyu 有什么特别？→ **没有独立视觉编码器**，直接把图像 patch 线性投影成 token 送进语言模型，省掉视觉塔、也省掉桥接层，代价是需要更强数据、分辨率灵活性靠 patch 切分。

A2 ★★★ LLaVA、BLIP-2、Flamingo 三者的桥接设计有什么本质区别？

A：

	桥接方式	视觉 token 数量	训练成本	核心思想
LLaVA	2 层 MLP（线性投影）	**不压缩**（与 patch 数相同）	最低（只训 MLP）	视觉塔特征已够好，简单映射即可
BLIP-2	**Q-Former**（32/64 个可学习 query + cross-attention）	**固定**（32/64）	中（两阶段多任务）	压缩 + 强对齐
Flamingo	Perceiver Resampler + **LLM 里插 gated cross-attention**	固定（resampler 输出）	中（插入新层）	不改 LLM 主干权重，只加"旁路"

本质区别一句话：LLaVA 是"**改维度**"，BLIP-2 是"**压数量 + 强对齐**"，Flamingo 是"**在 LLM 内部做融合**"。

追问：为什么 LLaVA 敢用最笨的 MLP？→ 因为它冻结双塔、数据量大（数十万~百万级指令），MLP 已经足够；而且论文消融显示，在冻结设定下复杂桥接的收益不显著。**这是"简单方案 + 足够数据"胜过"精巧结构"的典型案例。**

A3 ★★★ 视觉塔为什么大多用 CLIP/SigLIP 而不是 ImageNet 预训练的 ResNet/ViT?

A：三条理由：

1. **语义空间已经和文本对齐**：CLIP 系在 4 亿级图文对上做过对比学习，特征里已经含"可被语言描述"的语义；ImageNet 特征是判别 1000 类的，语义粒度不匹配。
2. **冻结就能用**：CLIP 特征在冻结条件下零样本泛化好，适合 VLM"冻结视觉塔"的训练范式。
3. **ViT 结构同构**：patch token 序列可以直接送进 LLM，不需要额外适配。

什么时候不用 CLIP：需要**密集预测**（分割/深度/占用）时，CLIP 的全局对齐特征反而丢失细粒度空间信息，这时 DINOv2（自蒸馏，保留空间结构）或纯监督 ViT 更合适。**混合做法**：CLIP 塔给语义 + DINOv2 给结构，双塔拼接。

追问：SigLIP 比 CLIP 好在哪？→ 用 pairwise sigmoid 替代 softmax 交叉熵，**不需要全 batch 归一化**，因此 batch 可以小、能分布式异步训练，更省内存；效果同级甚至更好，是现在很多开源 VLM 的默认视觉塔。

A4 ★★ BLIP 的 CapFilt 是干什么的?

A：网页图文对（alt-text）噪声大——图里是"一只狗在草地上"，alt-text 却是"点击购买"。CapFilt = **Captioner + Filter**：

1. **Captioner**：用图像生成 caption（合成描述）；
2. **Filter**：用 ITM 头判断"图文是否匹配"，把原 alt-text 和合成 caption 里不匹配的过滤掉。

本质是用模型给自己造更干净的监督（bootstrapping）。这条和现在"用强 VLM 重新 caption 训练数据"（ShareGPT4V 一类）是同一思想。

A5 ★★ ALBEF 的三个训练目标是什么？为什么需要 ITM?

A：**ITC（图文对比）+ ITM（图文匹配）+ MLM（掩码语言建模）**。

- **ITC** 学全局粗对齐（快、但只能到"整体像不像"）；
- **ITM** 用融合后的特征做二分类（细粒度、但要负样本，靠 ITC 的相似度做 **hard negative 挖掘**）；
- **MLM** 让模型学词级与图像区域的对应（补细粒度）。

为什么需要 ITM：ITC 用的是各自独立的编码（没有跨模态交互），学不到"这个词对应这块区域"；ITM 让图文进同一个编码器交互。**代价是推理时每对都要过一次融合编码器，慢——所以 BLIP-2 之后就用生成式目标替代了它。**

A6 ★★ CoCa 和 BLIP 的思路差异?

A：CoCa 把"**对比学习**（对齐）"和"**caption 生成**（生成）"放在**一个模型的两个头**上，共享图像编码器：一个头做图文对比（学判别式表征），另一个头做自回归 caption（学生成能力）。

意义：**证明对齐与生成可以共存且互补**（对比目标提供稳定的语义空间，生成目标提供细粒度语言监督）。现在很多 VLM 训练都是这个混合思路的延伸。

A7 ★★★ 视觉 token 到底有多少？为什么它是 VLM 的"预算中枢"?

A：算一遍就有体感： - 336×336、patch14 → 24×24 = **576 token** - 448×448、patch14 → 32×32 = **1024 token** - LLaVA-1.6 的 anyres（4 切片 + 1 缩略图）→ 约 **2880 token** - InternVL 的 pixel shuffle（2×2 拼通道）→ token **÷4**

而一张 4096 上下文的 LLM，一张高分辨率图就能吃掉 70%。所以：**分辨率（看得清）↔ token 数（算力/显存）↔ 上下文（能塞几张图/多轮）三者是硬约束的三角**。

工程手段：pixel shuffle、池化、Q-Former 压缩、动态分辨率（按需给 token）、只取关键帧（视频）。

A8 ★★ Qwen-VL 的 M-RoPE 解决什么问题?

A：普通 RoPE 是 1D 的（按序列位置旋转），但视觉 token 有 **时间（帧）+ 高 + 宽** 三维结构。M-RoPE（Multimodal RoPE）把旋转维度切成三份，分别编码 t/h/w，于是： - 同一帧内相邻 patch 的相对位置正确； - 不同帧之间有时间相对关系； - 图像与文本混排时位置语义不乱。

意义：让一个模型同时处理**图像、多图、视频**而不需要改结构——这是"原生多模态位置编码"的代表做法。

A9 ★ InternVL 的 pixel shuffle 为什么能省 token?

A：把相邻 2×2 个 patch 在**通道维**拼起来：原本 4 个 patch 各 `d` 维 → 合成 1 个 patch `4d` 维。token 数 ÷4，再用一个 MLP 把 `4d` 投回 `d`。

代价：空间分辨率信息被"压缩进通道"，细粒度定位会略损；但配合**动态分辨率**（大图切块）能补回来。这是"用通道换序列长度"的典型取舍。

A10 ★★ MiniGPT-4 和 LLaVA 的差别是什么?

A：结构上 MiniGPT-4 用 **Q-Former + 单层线性投影**（沿用 BLIP-2 的视觉侧），LLaVA 用 **CLIP ViT + 2 层 MLP**。数据上 MiniGPT-4 用短 caption 预训练 + 高质量长对话微调（用 GPT-4 生成、人工清洗），LLaVA 用 GPT-4 生成的视觉指令数据。

历史意义：MiniGPT-4 证明了"冻结 LLM + 轻量桥接"能获得对话式视觉能力；LLaVA 把这条路线做到了极致简化 + 规模化，成为开源 VLM 的事实标准。

A11 ★ 原生多模态（Early fusion）和"桥接式"到底差在哪?

A：差在**融合发生的位置**和**训练方式**：

- 桥接式：视觉塔独立编码（可能冻结）→ 投影 → 拼进 LLM 序列。融合发生在 LLM 的第一层之后（视觉 token 已经是"外来词"）。
- 原生式：图像 patch 直接当 token，与文本 token 从第一层起就在同一序列里做 attention。**模型从一开始就不知道"这是图像"还是"这是文本"**，全靠数据学。

代价：原生式通常要从头预训练（或大规模继续预训练），数据门槛极高；好处是**交错图文（多图穿插在长文档里）**天然支持，且不会出现"视觉塔与 LLM 表征错配"。

A12 ★ 视觉塔参数量要不要大？为什么很多模型用 ViT-L 而不是 ViT-H/G?

A：这是**性价比**问题。视觉塔只在 prefill 时跑一次，但它的计算量与**分辨率平方 × 参数量**挂钩；而 LLM 的算力主要在 decode（逐 token）。所以在"视觉塔够用"的前提下，加参数到视觉塔的边际收益远低于加在 LLM 上。

实践：ViT-L（约 300M）是主流；追求更强视觉理解时才上 ViT-H/SoViT；端侧反过来会用 ViT-S/B 配蒸馏。

A2 训练范式与数据（A13–A22）

A13 ★★★ VLM 训练为什么必须分阶段？一次性端到端训行不行?

A：能训，但**代价大且不稳**，原因三条：

1. **初始化错配**：随机初始化的桥接层输出是噪声，若同时解冻 LLM，这些噪声会通过梯度污染 LLM 的预训练能力（灾难性遗忘）。
2. **学习率不可共享**：视觉塔需要小 lr（保护预训练特征），桥接层需要大 lr（从零学），LLM 需要中 lr。分阶段可以分别设。
3. **数据配比不同**：对齐阶段用海量短 caption（学"图上有啥"），指令阶段用少量高质量对话（学"怎么答"）。混在一起训，模型会偏向数据量大的那类。

标准做法：阶段一冻结双塔只训桥接（对齐）→ 阶段二解冻 LLM 做指令微调 → 阶段三（可选）偏好对齐。

A14 ★★★ 对齐阶段（stage-1）为什么要冻结 LLM?

A：三个原因：

1. **保护语言能力**：桥接层初始输出没有语义，让 LLM 去适配它 = 让 LLM 学"如何理解噪声"，会破坏语言先验。
2. **训练信号更干净**：冻结 LLM 后，梯度只能改桥接层，等于在解"如何把视觉特征翻译成 LLM 能懂的词"这个子问题。
3. **成本**：只训几百万参数，单卡就能跑。

追问：那为什么有些模型（如 InternVL）在阶段一就解冻部分 LLM？→ 数据量足够大时（亿级图文对），直接联合训练能获得更强的对齐；小数据下必须冻结，否则一定崩。

A15 ★★ 指令微调数据怎么构造？有哪些来源？

A：五类来源，按质量排：

1. **人工标注**（最贵）：真实用户的问答、专家标注。
2. **强模型蒸馏**：用 GPT-4V/Claude 对图像生成多轮对话，再人工抽检。**注意只让它生成文本，不要用它生成图像**（版权与分布问题）。
3. **现有数据集改造**：VQA、检测（转成"图里有什么/在哪"）、OCR、grounding、文档理解，全部改写成对话格式。
4. **合成负样本**：故意问图中没有的东西，教模型说"没有"——**这是抑制幻觉最有效的数据手段**（对应 POPE 的 adversarial 设置）。
5. **纯文本数据混入**：防语言能力退化（见 A21）。

质量控制：去重（图像近重复是评测泄漏的头号来源）、答案长度控制（过长的回答会教模型"啰嗦"）、任务配比（OCR/推理/描述/grounding 要有比例）、难度分层。

A16 ★★ 怎么定图文 token 比例 / 数据配比？

A：没有普适公式，但有三条经验：

1. **按任务定 token 预算**：OCR/文档/grounding 需要高分辨率（给多 token），日常对话可以压缩（少 token）。
2. **按能力短板调**：评测发现哪块弱就加哪块数据（不是加权采样，而是补数据）。
3. **按阶段定比例**：对齐阶段图文对占绝对多数（可以 100%）；指令阶段通常图文 : 纯文本 ≈ 7:3 或 8:2。

监控手段：分别报"纯文本 benchmark"和"多模态 benchmark"，看是否出现"多模态涨、纯文本掉"的跷跷板。

A17 ★★ 分辨率策略有哪几种？各自代价？

A：四种：

策略	做法	token 代价	代表
固定分辨率	统一 resize 到 336/448	固定（576/1024）	LLaVA-1.5
动态切块（anyres）	大图切 N 块 + 1 张缩略图	N×576 + 576	LLaVA-1.6、InternVL
原生分辨率	按原分辨率切 patch，不缩放	与像素数成正比	NaViT、Fuyu
像素重排	2×2 patch 拼通道	÷4	InternVL

关键工程点：切块时块与块之间要**留重叠**（避免边界目标被切断），并做**位置编码的连续性处理**（否则模型不知道第 2 块在右边）。

A18 ★★ 多图 / 多轮对话在训练时怎么组织？

A：三条：

1. **特殊 token 标界**：每张图用 `<image>` / `<|image_pad|>` 包裹，模型靠这个知道"这里换了一张图"。
2. **图像只在首次出现时插入**：多轮里后续轮次引用"上面那张图"，不重复插 token（否则上下文爆炸）。
3. **loss mask**：只在 assistant 回复上算损失，用户提问和历史回复 mask 掉。

追问：多轮时历史图像 token 要不要丢？→ 上下文够就保留；不够时的常见策略是"旧图只留缩略图 token"或"旧图换成语义摘要"，本质是**压缩历史**。

A19 ★ 课程学习（curriculum）在 VLM 训练里怎么用？

A：由易到难排数据：短 caption → 长描述 → 单轮 QA → 多轮对话 → 需要推理/多图对比的难题。好处是**收敛更稳**（早期不被难题的噪声主导）、**避免早期过拟合到长答案格式**。

实践中最常见的"隐式课程"就是：对齐阶段全是短 caption（简单），指令阶段才上复杂任务。

A20 ★★ 合成数据有什么风险?

A：四个风险：

1. **分布坍缩**：全用强模型生成，学生会继承教师的偏好和错误（教师也可能幻觉）。
2. **风格单一**：句式、长度、格式趋同，模型失去多样性（用户会觉得"答得像机器人"）。
3. **事实污染**：教师对图像看错的地方，被当作"标准答案"教下去。
4. **评测污染**：合成时如果不小心用到评测集图像，指标虚高。

缓解：合成 + 人工抽检、保留一部分真实数据、拒绝采样（best-of-n 后人工/规则筛）、对合成数据降权。

A21 ★★ 混合模态训练时，怎么防止纯文本能力退化?

A：三条措施：

1. **数据层面保留纯文本**：图文:纯文本通常 7:3 ~ 8:2；SFT 阶段尤其重要。
2. **参数层面限制漂移**：用 LoRA 或只解冻部分层，减少对语言能力的破坏。
3. **评测层面同时盯**：每次报多模态分的同时报纯文本分，出现"跷跷板"就调整配比。

机制解释：图文数据的答案分布（描述性、短句）与文本数据（推理、长文）差异大，模型会向数据多的那侧偏——这叫**模态偏置**，不是简单的"遗忘"。

A22 ★★ 训练时视觉塔解冻有什么风险？什么时候该解冻?

A：**风险**：① 视觉塔参数量大，解冻后显存和算力翻倍；② 小数据下会破坏预训练特征（退化成"适配指令数据的私有特征"）；③ 与 LLM 的学习率不匹配时容易震荡。

该解冻的情况：领域差异极大（医学影像、红外、深度图、遥感）、需要极细粒度定位、数据量足够大（百万级以上）。**通常做法**：只解冻最后 1-2 层 + 小学习率（1/10），或加 LoRA。

A3 能力边界与失效模式（A23–A30）

A23 ★★★ VLM 的能力短板主要集中在哪五处?

A:

短板	表现	根因
细粒度计数	数错人数/物体数	对比学习不建模数量；token 聚合丢个体
空间关系	"左边的红球"搞错	全局对齐不建模相对位置；位置编码在视觉侧弱
精确 OCR	小字、密集文本读错	分辨率不足 + patch 太粗
视觉推理链	多步推理靠语言先验猜	训练数据里缺"看图推理"的中间步骤
grounding 精度	框偏、指代歧义	缺区域级监督；token 与像素的对应不唯一

面试加分：每说一个短板，配一句"所以工业上怎么绕"——如 OCR 靠 dynamic tiling + 专门 OCR 数据，空间关系靠显式 3D/2D 坐标输入（SpatialVLM 一类）。

A24 ★★★ 幻觉的三种成因在模型结构上分别对应什么?

A:

成因	结构层面的解释	缓解手段
语言先验主导	LLM 占了绝大部分参数，视觉 token 在序列里是"少数派"，attention 容易忽略它们	提高视觉 token 权重/数量、对比解码、加大视觉分辨率

成因	结构层面的解释	缓解手段
视觉信息不足	压缩过度（Q-Former/池化）、分辨率低 → 视觉证据本身弱	减少压缩、dynamic tiling、多图交叉验证
对齐错位	桥接层是"翻译器"，翻译错了 LLM 就会基于错误信息自信作答	对齐阶段加更多数据/多任务监督、区域级对齐

追问：为什么"加更多指令数据"不能根治幻觉？→ 指令数据教的是"怎么答"，不改变"看不看得清"；幻觉主要是**视觉证据强度**问题，得从分辨率/token/监督入手。

A25 ★★ 为什么 VLM 在计数上特别差？

A：四个原因叠加：

1. **训练目标不鼓励逐个体建模**：对比学习/生成目标都没有"必须数对"的压力。
2. **token 聚合丢失个体性**：patch 特征池化成全局语义时，5 个人和 6 个人的差异被平均掉。
3. **位置编码不显式表示"第几个"**。
4. **数据偏差**：训练数据里计数类问答少且答案分布偏（多半是"1 个""2 个"）。

能改善的做法：专门的计数数据 + 让模型输出检测框/点（"先定位再数"）+ 高分辨率。

A26 ★★ 空间关系理解为什么难？

A：因为 VLM 的语言侧对"左右上下"极敏感（文本里这两词很常见），但视觉侧**没有显式坐标系**——patch 序列的位置编码是 2D RoPE 或可学习编码，与"图像平面坐标系"不是一回事。于是模型倾向于**根据语言先验猜**（"盘子通常在下"），而不是看图。

工业解法：把坐标显式输入（"物体 A 在 (x1,y1)-(x2,y2)"）、加 3D/2D 位置监督数据、或干脆让模型先输出框再回答。

A27 ★★ OCR 能力怎么系统性提升？

A：四件事，按性价比排：

1. **分辨率**（最关键）：dynamic tiling 把长文档切成条带，每块单独编码。
2. **数据**：大量文档/表格/截图类 QA（DocVQA、ChartQA、TextVQA 一类），以及纯 OCR 转录任务。
3. **视觉塔**：patch 更小或专用 OCR 前端。
4. **后处理**：让模型输出结构化结果（JSON/表格），便于校验。

A28 ★★ VLM 做视觉 grounding 的三种输出形式？

A：① **文本坐标**（"`<box>(x1,y1,x2,y2)</box>`"）——最通用，但精度受 token 化影响（数字被切成多个 token）；② **特殊 token / 离散 bin**（把坐标量化成 bin 分类）——精度可控；③ **专用 head**（接一个检测头输出框/mask）——精度最高，但破坏了"纯语言模型"的统一性。

趋势：主流通用模型用 ①，专门的方向（指代分割、GUI agent）用 ②③。

A29 ★ 视频理解比图像多出来的难点是什么？（三条）

A：① **token 爆炸**（帧数 × 每帧 token）；② **时序建模**（帧间关系、事件先后、因果）；③ **长视频检索**（1 小时视频不可能全塞，必须"摘要 + 检索"）。附加难题：视频数据标注昂贵、评测集难做（时序问答的答案歧义大）。

A30 ★★ 为什么说"VLM 的数学推理很多时候是在做语言推理"？

A：因为它可以从题目文本推出答案（哪怕图没看对）。诊断方法很直接：**把图像遮挡/替换成无关图像，看答案是否改变**——不变就说明它没用图。这也是 HallusionBench 这类"视觉依赖测试"的设计思想。

A4 评测体系（A31–A38）

A31 ★★★ 主流评测集各自测什么？给一张能背的表。

评测集	测什么	需要记住的细节
MMBench	综合能力（20 个能力维度）	**CircularEval**：选项循环移位，全对才算对，消除位置偏置
MMMU	大学水平学科推理（图、表、公式、考试题）	多学科多图，考专业知识，人类专家也只有 ~80%
SEED-Bench	生成式理解（图+视频）	多项选择，人工校验过
POPE	**物体幻觉**	random / popular / **adversarial** 三设置；报 F1 和 yes 比例
HallusionBench	幻觉（视觉依赖 + 反事实）	专测"看图 vs 靠先验编"
MM-Vet	综合（识别+推理+OCR 交叉）	GPT-4 打分，看能力维度组合
RefCOCO/+/g	指代定位 grounding	Acc@0.5（IoU>0.5）或点命中率
TextVQA / DocVQA / ChartQA / OCRBench	OCR 与文档	工业落地最常看这四个
MathVista	数学视觉推理	图表+几何+公式
VideoMME / LongVideoBench	视频（短/中/长，有无字幕）	必须**分时长档**报，长视频普遍崩

A32 ★★★ CircularEval 是什么？为什么必须有？

A：把一道多选题的选项**循环移位 N 次**（N = 选项数），每次让模型作答，**N 次全对才算答对**。

原因：VLM 对选项位置有强烈偏置（训练数据里 A/B 出现频率高、或 token 化后某些位置的概率先验高），不移位会系统性高估能力——有研究报告这个偏差能贡献 10+ 个点的虚高。

面试意义：讲得出 CircularEval，等于告诉面试官"我读过评测实现，不只看排行榜数字"。

A33 ★★ POPE 的三个设置为什么要区分？

A：因为负样本的难度决定评测的可信度：

- **random**：随机挑不存在物体 → 太简单，模型靠"这词常见"就能答对；
- **popular**：挑数据集中高频但图里没有的物体 → 中等；
- **adversarial**：挑**与图中物体共现频率高**的物体（有椅子就问"有桌子吗"）→ 真考验"是否在看图"。

只看 random 分数会严重虚高，三个设置一起报才有意义。

A34 ★★ 评测里最容易自欺的四个地方？

A：① **数据泄漏**（训练图和评测图近重复，CLIP 系预训练数据几乎覆盖全网）；② **prompt 敏感**（换模板差几个点，不报 prompt 就是不完整）；③ **选项偏置**（不做 CircularEval）；④ **只报总分**（不报分项，OCR 强推理弱看不出来）。

加分补一句：还要报 **解码配置**（贪心/beam/温度）和**是否 few-shot**——这些不写，结果不可复现。

A35 ★★ 幻觉怎么量化？除了 POPE 还有什么办法？

A：四类方法：

1. **判别式探测**（POPE）：问"有没有 X"，看 yes/no 是否正确 → 简单、可自动化；
2. **生成式一致性**：同一张图问多次/换问法，答案不一致即为不稳定（self-consistency 视角）；
3. **视觉依赖测试**：把图替换成空白/无关图，答案不该变（该变才说明用了图）；
4. **人工/强模型打分**：MM-Vet 式，成本高但是最终裁判。

A36 ★★ grounding 的评测为什么比检测的 mAP 更"宽容"?

A：因为 grounding 的目标是"**指代唯一性**"而不是"检测全面性"：输入是一句描述，只要求找到**那个**目标，不惩罚漏检其他物体。指标常用 Acc@0.5 或点命中（点落在 GT mask 内），后者更宽容（不要求框贴合）。

难点在指代歧义：图里有三只狗，"那只左边的狗"需要理解关系，而不是靠类别先验。

A37 ★ 长视频评测为什么必须分档报?

A：短视频（<2 分钟）靠帧采样就能覆盖，模型表现接近图像；中长视频开始依赖时序推理与检索，性能断崖式下降。**只报平均分会掩盖这个断崖**——所以 VideoMME 明确分短/中/长三档，并区分有无字幕（有字幕时模型可能"读字幕"而不看画面，这也是要看分项的原因）。

A38 ★★ 怎么评测"模型有没有真在看图"？设计一个实验。

A：三个可落地实验：

1. **图像替换**：换成同类无关图 → 答案应显著改变（不变 = 没用图）。
2. **局部遮挡**：遮住关键区域 → 答案应变差（不变 = 没依赖该区域）。
3. **反事实图像**：把"红色苹果"改成蓝色 → 答案应跟着变（不变 = 靠先验）。

这三个也是 HallusionBench 的核心思想。**面试时能讲出实验设计，比背分数有用。**

A5 推理与部署（A39–A45）

A39 ★★★ VLM 推理的延迟构成？哪一段是瓶颈?

A：四段：① **预处理**（resize/切块/归一化，CPU 上常被忽略但很贵）；② **视觉塔前向**（与分辨率平方相关）；③ **prefill**（把图文 token 一次前向，计算密集）；④ **decode**（逐 token，访存密集，与输出长度线性）。

瓶颈判断： - 高分辨率 + 短回答 → 卡在 ②③（视觉塔 + prefill）； - 低分辨率 + 长回答 → 卡在 ④（decode，KV cache 带宽）。

这句话很值分："端侧 VLM 要先分清是 prefill-bound 还是 decode-bound，两者的优化手段完全不同。"

A40 ★★ VLM 的 KV cache 和纯文本 LLM 有什么不同?

A：机制一样（缓存每层 K/V），但 VLM 有两处特殊：

1. **视觉 token 的 KV 是固定的**：同一张图多轮问答时，视觉部分的 KV 可以**跨轮复用**（prefix cache 思路）——这是多模态特有的优化收益。
2. **视觉 token 数量大且不等于文本长度**：一张 anyres 图可能 2880 token，KV cache 直接翻几倍，长多轮时显存爆得比纯文本快得多。

A41 ★★ 图像 token 太多，有哪些工程压缩手段?

A：五类：① 视觉侧压缩（pixel shuffle、池化）；② 桥接层压缩（Q-Former/resampler）；③ token 剪枝（按注意力重要性保留 top-k）；④ 动态分辨率（按任务给 token）；⑤ 缓存复用（同一图的多轮只算一次）。

面试提醒：压缩一定损精度，所以要说清"在哪一层压、损失什么"——通道压缩损细节，序列压缩损定位，两者可叠加但要注意叠加放大误差。

A42 ★★ VLM 量化有什么特别的坑?

A：三个：① **视觉塔对量化更敏感**（输入分布极端、动态范围大），常见做法是视觉塔保 FP16、只量化 LLM；② **视觉 token 的离群值**（ViT 输出常有较大激活）会破坏 INT8，需要 per-channel 或对视觉通路单独处理（类似 AWQ 保护重要通道）；③ **KV cache 量化**在多轮/长上下文里收益最大，但要单独校准。

A43 ★ 投机解码能用在 VLM 上吗?

A：能，但草稿模型的输出分布要覆盖**图文场景**——用纯文本小模型做草稿，遇到需要"看细节"的问题命中率会掉。实践中：① 用同系列的小 VLM 作草稿；② 或者只在"回答格式固定/重复性高"的场景（如结构化抽取）用。

A44 ★★ 端侧 VLM 的现实约束是什么?

A：四条硬约束：① **内存**（权重 + KV cache + 激活，手机级别只有几 GB 可用）；② **算子支持**（动态形状、动态分辨率、attention 变体在 NPU 上常常要改）；③ **功耗/发热**（长时间推理会降频）；④ **预处理开销**（切块/resize 在 CPU 上可能比模型还慢）。

你的接口（可直接讲）："我在 MDC610 上做过 ONNX→ATC/AOE 的完整链路，所以端侧 VLM 我会先问三件事——**目标 SoC 的算子白名单、内存预算、是否允许动态分辨率**。"

A45 ★ 昇腾/国产芯片上部署 VLM，主要工作量在哪?

A：四块：① **算子适配**（不支持的算子拆解或等价重写，例如动态 attention、top-k、动态 shape）；② **shape 固定化**（把动态分辨率按档位枚举，分桶推理）；③ **精度对齐**（fp16 溢出、算子融合顺序导致的数值差异，要逐层比对）；④ **内存规划**（模型段/工作空间手工分配，KVCache 动态增长需要预分配）。

Part B 机器学习算法原理

> 这一部分**不写代码**（代码在手撕两册里），只讲**定义、推导、性质、对比、失效场景**——面试官问"为什么"的时候用。

B1 Softmax 家族（B1–B8）

B1 ★★★ 写出 softmax，它的雅可比矩阵是什么?

A:

```
softmax(z)_i = exp(z_i) / Σ_j exp(z_j)
```

雅可比（这个能推出来很加分）：

```
∂p_i/∂z_j = p_i · (δ_ij − p_j)     即   J = diag(p) − p · pᵀ
```

含义：**对角线是 p_i(1−p_i)（自身对自身的敏感度），非对角线是 −p_i p_j（此消彼长）**，因为输出必须和为 1，抬高一个必然压低其他。

两个关键性质：1. **平移不变**：`softmax(z + c) = softmax(z)`（分子分母同乘 e^c）。 2. **和为 1 且恒正**：输出可解释为概率单纯形上的一点。

B2 ★★★ 为什么要减 max? 给出严格的数值论证。

A：`exp(x)` 在 float32 下 x > 88.7 就上溢成 `inf`，`inf/inf = nan`。减 max 后：

```
softmax(z)_i = exp(z_i − m) / Σ_j exp(z_j − m)，其中 m = max_j z_j
```

- **数学等价**：分子分母同乘 `e^{-m}`，结果不变（这是"等价"，不是近似）。
- **数值安全**：减完后最大项是 `exp(0)=1`，其余 ≤ 1，不会上溢；小项下溢成 0 也无害（softmax 里 0 就是"几乎不可能"）。
- **但下溢在 log 域有害**：如果先算 softmax 再取 log，下溢成 0 的地方会变 `-inf`。所以**交叉熵要直接在对数域算**（log_softmax / logsumexp），这是工程上的标准做法。

追问：全 −inf 的一行会怎样？→ 分母 0 → NaN。所以 mask 用 `-1e9` 而不是 `-inf`，或对全屏蔽行单独补 0。

B3 ★★★ 推导 softmax + 交叉熵的梯度为什么是 `p − y`。

A：设 `L = −log p_y = −z_y + log Σ_j exp(z_j)`。

对 `z_i` 求导，分两种情况：

```
∂L/∂z_i = −δ(i=y) + exp(z_i)/Σ_j exp(z_j) = p_i − δ(i=y)
```

即 **`∂L/∂z_i = p_i − y_i`**（y 是 one-hot）。

这个结果的意义：1. 梯度**不含 softmax 的雅可比**——两者精确抵消了（这也是框架把 softmax+CE 合成一个算子（如 `F.cross_entropy`）的原因：更快、更稳）。2. 误差越大梯度越大（预测 0.1、真值 1 → 梯度 −0.9），**没有饱和问题**；3. 对比 MSE + softmax：后者梯度多一个 `p(1−p)` 因子，在"预测 0.99 但真值是 0"这种最该纠错的情况下梯度趋于 0。

B4 ★★ softmax 的温度参数有什么作用?

A：`softmax(z/T)`：

- `T = 1` 标准；
- `T > 1` → 分布更平（熵更大），用于**知识蒸馏**（软标签含"类间相似性"信息）、**采样多样性**、**LLM 生成**；
- `T < 1` → 分布更尖（接近 argmax），用于确定性输出；
- `T → 0` → 退化成 one-hot（hard max）；`T → ∞` → 均匀分布。

为什么蒸馏要用 T：教师模型的原始 logits 里，"猫"和"狗"的相似性会被 softmax 压成接近 0 和 1，T 放大后这部分"暗知识"才显现出来，学生才能学到。

B5 ★★ softmax 存在"过参数化"，怎么理解?

A：`softmax(z + c·1) = softmax(z)`——**所有 logit 同加一个常数，输出不变**。所以：- C 类分类实际只有 **C−1 个自由度**，参数矩阵存在一整个方向的冗余（解不唯一）；- 导致 Hessian 奇异、优化不唯一、某些正则化行为异常。

三种处理：① 固定某一类的 logit 为 0（等价于用 C−1 组参数，这在数学上恰好等价于 sigmoid 二分类的推广）；② 加 L2 正则（把解拉向唯一）；③ 干脆不管（深度学习里因为正则/优化噪声，实践中不影响）。

B6 ★★ softmax 和 sigmoid 是什么关系?

A：**二分类时两者等价**（差一个常数偏移）：

```
softmax([z, 0])_1 = e^z/(e^z+1) = σ(z)
```

所以"softmax + CE"在 C=2 时与"sigmoid + BCE"给出相同的决策边界和梯度形式。

关键区别在多标签：如果一个样本可以同时属于多个类（不互斥），必须用**逐元素 sigmoid + BCE**，因为 softmax 强制各类概率和为 1（互斥假设）。

B7 ★★ log_softmax 与 logsumexp 技巧是什么?

A：

```
log_softmax(z)_i = z_i − logsumexp(z),   logsumexp(z) = m + log Σ_j exp(z_j − m)
```

- 直接算 `log(softmax(z))` 会先下溢再取 log → `−inf`；log_softmax 全程在对数域，**数值稳定**。
- `logsumexp` 是核心恒等式：**先减最大值，算完再加回来**。它出现在交叉熵、CRF、变分推断、混合模型里。
- 工程上 `torch.logsumexp` 已内置该实现；手写时**务必自己减 max**。

B8 ★ softmax 在什么情况下梯度会消失? 怎么缓解?

A：当某个 logit 远大于其他（`z_max − z_second ≫ 0`）时，softmax 输出接近 one-hot，`p_i(1−p_i) → 0`，**梯度饱和**。这在训练早期 logit 尺度过大时容易出现（初始化不好、学习率过大、输入未归一化）。

缓解：① 输出层用小初始化（或用温度缩放）；② 加 label smoothing（防止一路把概率推到 1）；③ 用 focal loss（主动放大难例权重）；④ 检查输入特征尺度与归一化。**这也是"训练初期 loss 不动"的常见原因之一。**

B2 信息熵与信息论（B9–B16）

B9 ★★★ 信息量和熵的定义与直觉?

A:

- **自信息**：$I(x) = -\log p(x)$。概率越小，事件发生带来的"信息量"越大（小概率事件更有信息）。
- **熵**（信息熵）：$H(X) = -\Sigma_x p(x) \cdot \log p(x) = E[-\log p(X)]$，即**平均信息量 / 不确定度**。
- 单位：log 以 2 为底是 bit，以 e 为底是 nat（深度学习多用 nat）。
- **极值**：均匀分布熵最大（最不确定）；确定性分布熵为 0。

一句话直觉：熵 = "描述这个随机变量平均需要多少比特" = "我猜它有多难"。

面试常配的追问：为什么熵是不确定度的合理度量？→ 三条公理（连续性、均匀分布最大、独立可加）唯一确定出 $-\Sigma p \log p$ 的形式（Shannon 唯一性定理）。

B10 ★★★ 联合熵、条件熵、链式法则?

A:

```
联合熵    H(X,Y) = −ΣΣ p(x,y) log p(x,y)
条件熵    H(Y|X) = H(X,Y) − H(X)          （给定 X 后 Y 的剩余不确定度）
链式法则  H(X,Y) = H(X) + H(Y|X)
```

重要性质：- $H(Y|X) \leqslant H(Y)$：知道 X **不会增加** Y 的不确定度（等号当且仅当独立）。- $H(X,Y) \leqslant H(X) + H(Y)$：等号当且仅当独立。

在树模型里怎么用：ID3 的信息增益本质就是 $H(D) - H(D|A)$，即"用特征 A 划分后，标签的不确定度减少了多少"。

B11 ★★★ 互信息是什么？有什么意义?

A:

```
I(X;Y) = H(X) − H(X|Y) = H(Y) − H(Y|X) = H(X) + H(Y) − H(X,Y)
```

含义：知道 Y 能减少多少关于 X 的不确定性（对称的"相关性"度量）。

三个用途（面试可举）：1. **决策树的信息增益** = $I(\text{标签};\ \text{特征})$；2. **特征选择**：互信息高 = 该特征与标签相关性强；3. **多模态对齐的另一种视角**：CLIP 的 InfoNCE 本质在最大化**图像与文本的互信息下界**（这是理解对比学习的信息论视角）。

B12 ★★★ KL 散度的定义与三个性质?

A:

```
D_KL(P||Q) = Σ_x p(x) · log( p(x)/q(x) )
```

三个性质：1. **非负**：$D_{KL} \geqslant 0$，等号当且仅当 P=Q（由 Jensen 不等式或 Gibbs 不等式）。2. **非对称**：$D_{KL}(P||Q) \neq D_{KL}(Q||P)$，所以**不是距离**（不满足三角不等式）。3. **不度量"距离"而度量"代价"**：$D_{KL}(P||Q)$ 是"用 Q 去编码 P 的额外代价"。

非对称的实际后果（这个例子很值分）：设 P 是双峰分布，Q 是单峰高斯：- **前向 $KL(P||Q)$**（最小化 Q 覆盖 P 的所有模式）→ Q 会摊平成宽分布，**mass-covering**；- **反向 $KL(Q||P)$**（Q 避开 P 概率为 0 的地方）→ Q 会塌到某个峰，**mode-seeking**。这就是变分推断（前向）与 GAN/反向 KL 行为差异的根源。

B13 ★★★ 交叉熵和 KL 散度的关系？为什么分类要用交叉熵?

A:

```
H(P,Q) = −Σ p log q = H(P) + D_KL(P||Q)
```

即 **交叉熵 = 真实分布的熵 + KL 散度**。因为 $H(P)$ 与模型参数无关（数据固定），所以：

最小化交叉熵 ≡ 最小化 KL 散度，即让预测分布尽量逼近真实分布。

用交叉熵而不用 KL 的工程原因：**不需要知道 H(P)**（P 是 one-hot 时 H(P)=0，直接就是 −log q_y），计算更简单。

一句话：交叉熵是"用模型分布去编码真实标签的代价"，最小化它就是最大似然估计。

B14 ★★★ 信息增益怎么算？它偏向什么？

A：

```
Gain(D, A) = H(D) − Σ_v (|D_v|/|D|)·H(D_v) = H(D) − H(D|A) = I(D; A)
```

即"用特征 A 划分后标签熵的下降量"，也就是互信息。

偏向多值特征：特征取值越多，划分得越细，`H(D|A)` 越容易小（极端情况：用 ID 号划分，每个样本一组，增益最大但毫无泛化）——**这是 ID3 的根本缺陷**，被 C4.5 的增益率修正。

B15 ★★ 信息增益率与固有值是什么？为什么能修正偏向？

A：

```
增益率 = Gain(D, A) / IV(A),   IV(A) = −Σ_v (|D_v|/|D|)·log(|D_v|/|D|)
```

`IV(A)` 是特征 A 自身的**固有值/分裂信息**（把样本按 A 分成 v 组的"熵"），取值越多 IV 越大，于是增益率被惩罚。

但增益率也有反向偏向（偏向取值少的特征），所以 C4.5 用的是**启发式**：先找出增益高于平均的特征，再在其中选增益率最高的。

B16 ★★ 基尼指数和熵是什么关系？为什么 CART 用基尼？

A：

```
基尼    Gini = 1 − Σ_c p_c²   =  Σ_{c≠c'} p_c p_{c'}     （随机取两个样本、类别不同的概率）
熵      H = −Σ_c p_c log p_c
```

- **形状高度相似**：都在均匀分布处取最大，都在纯节点处取 0；二分类下 `Gini = 2p(1−p)`，与熵的曲线几乎重合。
- **基尼是熵的近似**（在均匀分布附近可用低阶展开近似），所以划分结果基本一致。
- **用基尼的原因**：只有乘法加法、**没有 log**，计算快（大数据下差异明显）；且梯度更"平缓"，对噪声略鲁棒。

追问：两者的最优划分会不同吗？→ 可能不同，但实践差异很小；论文级对比显示 mIoU/准确率差异通常在 1% 内。

补充：还有 **最大熵原理**（在已知约束下选熵最大的分布，最"不假设"）和 **困惑度**（`perplexity = exp(H)`，语言模型常用），面试顺口提到很加分。

B3 分类树（B17–B28）

B17 ★★★ ID3 / C4.5 / CART 三种算法怎么对比？

A：

	ID3	C4.5	CART
划分准则	信息增益	**信息增益率**	**基尼指数**（分类）/ MSE（回归）
分裂方式	**多叉**（每个取值一支）	多叉	**二叉**
连续特征	不支持	支持（排序+二分）	支持
缺失值	不支持	支持（按权重分配到各分支）	支持（代理分裂 surrogate split）
剪枝	无	悲观剪枝（PEP）	**代价复杂度剪枝（CCP）**
输出	分类	分类	**分类 + 回归**

一句话："ID3 是原型，C4.5 补工程（连续/缺失/剪枝），CART 补统一性（二叉 + 回归）——sklearn 和所有 GBDT 家族用的都是 CART。"

B18 ★★★ 为什么 CART 用二叉树?

A：四个理由：

1. **避免碎片化**：多叉会让样本被迅速切碎（取值多的特征尤其），每个分支样本太少 → 过拟合且统计不可靠。
2. **表达力更强**：二叉允许**同一特征被多次使用**（"年龄<30 且 年龄>20"），而多叉一个特征只能用一次。
3. **计算高效**：每次只需找最优二分点，比多路划分的组合搜索简单得多。
4. **统一处理类别型特征**：把"取值集合"按 one-vs-rest 二分（对类别特征做分组），天然支持"把 {红,蓝} 归一边"。

B19 ★★★ 连续特征怎么处理?为什么只需枚举相邻中点?

A：做法：该特征的所有取值**排序去重** → 只在**相邻不同取值的中点**试切分。

为什么只需相邻中点：对同一特征，若两个阈值落在同一对相邻取值之间，**划分出的样本集合完全相同**（因为区间内没有样本），所以只有 n−1 个"有效"候选阈值（n = 去重后取值数），彼此等价。

复杂度：排序 $O(n \log n)$ + 一次扫描 $O(n)$（用前缀计数增量更新左右子集的不纯度），而不是"每个阈值重扫一遍"的 $O(n^2)$。

追问：为什么要去重？→ 相同取值的样本任何阈值都分不开，切开是无意义分裂。

B20 ★★★ 剪枝的三种方式?代价复杂度剪枝怎么工作?

A：

方式	做法	优缺点
预剪枝	提前停：深度上限、叶最小样本、增益阈值、纯度阈值	快；但**贪心视野短**，可能过早停止（当前增益小、后续收益大）
后剪枝	先长满，再自底向上判断"是否合并叶子"	更准（看到全树后再决策），但训练成本高
CCP（代价复杂度）	定义 $R_\alpha(T) = R(T) + \alpha \cdot \lvert T \rvert$（R 是训练误差，	T

CCP 的机制：α=0 时最优是全树；α 增大时叶子的"惩罚"变重，逐步剪枝 → 得到一串嵌套子树 → 用验证集选最好的 α（sklearn 的 `ccp_alpha`）。

面试话术："预剪枝省时但短视，后剪枝准但贵；工业上常见组合：先用预剪枝控深度（防止爆内存），再用后剪枝微调。"

B21 ★★ 缺失值怎么处理?

A：两种主流：

- **C4.5**：把缺失样本**按权重分到所有分支**（权重 = 各分支样本比例），增益计算时乘上"有效样本比例"的折扣因子。
- **CART（sklearn 不支持，但理论上有）**：**代理分裂（surrogate split）**——为每个主分裂再学若干"代理特征"，推理时若主特征缺失，用与它最相似的代理特征代替。

工程实践：多数框架直接要求先填充缺失值（均值/中位数/模型预测），再建树——因为树分裂的代理机制实现复杂且收益不稳定。

B22 ★★ 回归树怎么建?叶子输出什么?

A：和分类树完全同构，只换两处：

1. **划分准则**：从基尼换成 **MSE 下降**（$MSE = (1/n)\Sigma(y-\bar{y})^2$）。
2. **叶子输出**：从"多数类"换成 **均值**（最小化平方误差的最优常数就是均值；若损失是 MAE，则取**中位数**）。

注意：回归树的预测是**分段常数**（一个叶子一个值），所以它**不能外推**（超出训练范围的值只能给边界值），且对目标做单调变换会改变结果（因为 MSE 不等变）。

B23 ★★★ 特征重要性怎么算？有什么偏差？

A：两种：

1. **不纯度重要性（Gini/MSE importance，sklearn 默认）**：把"该特征每次分裂带来的加权不纯度下降 × 样本数"累加后归一化。
2. **置换重要性（permutation importance）**：打乱该特征的值（破坏其与标签的关系），看验证指标下降多少。**更可靠、更贵**。

三种已知偏差：- **偏高基数/多值特征**（分裂机会多，累积下降自然大）；- **对相关特征会"分摊"**（两个高度相关的特征各拿一半，看起来都不重要）；- **在不纯度重要性里，训练集上计算会偏向能拟合噪声的特征**。

面试加分："所以我一般用置换重要性做最终判断，不纯度重要性只用来看相对排序。"

B24 ★★ 决策树为什么不稳定？为什么要集成？

A：**不稳定**的原因：树的分裂是**贪心的离散选择**——数据微扰（换一批样本、加几个噪声点）可能让最优切分点改变，后面的整棵子树结构随之变化，预测方差大。

偏差-方差视角：单棵树**偏差低（可以拟合很复杂的边界）、方差高**。所以：- **Bagging / 随机森林**：并行多棵 + 平均 → **降方差**（不显著降偏差）；- **Boosting / GBDT**：串行拟合残差 → **降偏差**（同时要控制方差，所以用小树 + 学习率）。

B25 ★★★ RF / GBDT / XGBoost / LightGBM / CatBoost 怎么串起来？

A：

	集成方式	优化目标	关键创新
随机森林	Bagging（样本 + 特征双采样，并行，投票/平均）	降方差	双重随机性让树之间去相关
GBDT	Boosting（串行，每棵树拟合**负梯度/残差**）	降偏差	用树做梯度下降
XGBoost	GBDT + 工程	二阶泰勒 + 显式正则	二阶信息、正则化（$\gamma T + \lambda w^2/2$）、稀疏感知、特征级并行、近似分位点
LightGBM	GBDT + 效率	同 XGB	**leaf-wise 生长**（同一叶数下更低 loss，但易过拟合，需控 `num_leaves`）、直方图分桶、**GOSS**（保留梯度大的样本）、**EFB**（互斥特征捆绑）
CatBoost	GBDT + 类别特征	同 XGB	**对称树（oblivious）**、**ordered boosting**（解决目标泄漏）、类别特征的有序目标编码

一句话链："RF 是并行降方差，GBDT 是串行降偏差；XGBoost 给 GBDT 加了二阶和正则，LightGBM 把速度做到极致，CatBoost 专治类别特征。"

B26 ★★ XGBoost 相对传统 GBDT 的关键改进？

A：四件事：

1. **二阶泰勒展开**：目标函数用一阶梯度 g + 二阶 h 近似，**最优叶权重有闭式解** $w^* = -\Sigma g/(\Sigma h+\lambda)$，分裂增益也有解析式。这比只用一阶梯度收敛更快、更准。
2. **显式正则**：$\Omega = \gamma T + (\lambda/2)\Sigma w^2$，直接在目标函数里惩罚树复杂度（叶数 T 和叶权重 w），而不是靠早停/剪枝。
3. **稀疏感知**：为缺失值学一个默认方向（分裂时把缺失样本统一分到增益大的一侧）。
4. **工程**：特征预排序（block 结构）、特征级并行、近似分位点（`approx_hist`）、缓存优化。

B27 ★★ LightGBM 的 leaf-wise、GOSS、EFB 分别解决什么？

A：

- **leaf-wise（按叶生长）**：每次选"增益最大的叶子"继续分裂，而不是按层。同样的叶数下 loss 更低（效率高），但**容易长出深的不平衡树 → 过拟合**，所以要用 `num_leaves` 和 `max_depth` 双重约束。

- **GOSS（Gradient-based One-Side Sampling）**：梯度大的样本对增益贡献大，**保留全部大梯度样本 + 随机采样一部分小梯度样本**（并放大权重），用少量样本近似全量增益。
- **EFB（Exclusive Feature Bundling）**：稀疏场景下很多特征互斥（不会同时非零），把它们**捆绑成一个特征**，降低特征维度 → 减少直方图构建开销。

B28 ★ 什么时候选树模型，什么时候选线性/神经网络?

A:

场景	选树	选线性/NN
表格数据、中小规模、需要可解释	✅树（尤其 GBDT，仍是表格任务的王者）	—
高维稀疏（文本 TF-IDF）	—	✅线性（快、可解释）
特征有强交互、非线性	✅树	✅NN（需更多数据）
需要外推（预测超出训练范围）	❌树不能外推	✅线性/NN
需要平滑输出	❌分段常数	✅
图像/语音/文本原始信号	—	✅深度模型

一句话："树赢在'不需要特征工程 + 自动做交互 + 可解释'，输在'不能外推、不平滑、不稳定'。"

B4 K-Means（B29–B36）

B29 ★★★ KMeans 的算法与目标函数?

A:

目标函数（inertia / 簇内平方和）：

```
J(C, μ) = Σ_i min_j ||x_i − μ_j||²
```

算法（交替优化 / 坐标下降）：

```
1. 初始化 k 个中心
2. E 步（固定中心）：把每个点分给最近的中心
3. M 步（固定分配）：把每个中心更新为其簇内所有点的均值
4. 重复 2-3 直到中心移动量 < tol 或达到迭代上限
```

为什么 M 步是均值：固定分配后，`J = Σ_j Σ_{i∈C_j} ||x_i − μ_j||²` 对 μ_j 求导置零，得 `μ_j = (1/|C_j|)Σ x_i` ——即**均值是最小化平方误差的最优常数**。

B30 ★★★ 为什么 KMeans 一定收敛？为什么只是局部最优?

A:

收敛：E 步（在固定中心下，把点分给最近中心）不会增大 J；M 步（固定分配下取均值）也不会增大 J。J 单调不增且有下界 0；而划分方案是**有限个**（n 个点分到 k 个簇，最多 k^n 种，实际远少），所以不会无限循环 → **有限步内收敛到一个驻点**。

局部最优：因为 J 非凸，交替优化只保证到达**坐标下降的局部极小**。初始化不同 → 收敛到不同的局部解（我实测一组数据：20 个随机种子里有 7 个没能正确分出三个簇，最差的只对上 125/150）。

工程对策：`n_init` 多次随机初始化取 inertia 最小；或 k-means++ 初始化。

B31 ★★★ k-means++ 的原理？为什么有效?

A：**D^2 采样**：

```
1. 第一个中心均匀随机选一个点
2. 之后每个新中心按概率 ∝ D(x)² 采样（D(x) = 到最近已有中心的距离）
3. 重复到选出 k 个
```

即"**离已有中心越远的点，越容易被选为新中心**"。

为什么有效：随机初始化最坏情况可能把两个中心都放在同一个真实簇里（剩下一个簇分不到中心），D^2 采样让中心天然分散，规避了这种退化。

理论保证：k-means++ 的期望代价 ≤ **O(log k)** 倍最优解（对比随机初始化的任意差）。代价只是初始化时多算几次距离。

B32 ★★★ k 怎么选？（四种方法）

A：

1. **肘部法（elbow）**：画 inertia-k 曲线，找拐点（边际收益骤降处）。缺点是"肘"常常不明显。
2. **轮廓系数（silhouette）**：`s = (b − a)/max(a, b)`，a = 到本簇其他点的平均距离，b = 到最近其他簇的平均距离。范围 [−1,1]，越大越好，取 s 平均最大的 k。
3. **Gap statistic**：比较 log(inertia) 与"零假设下（均匀分布）的期望值"的差，差距最大处为最优 k。
4. **稳定性 / 下游指标**：多次子采样聚类的一致性；或直接用下游任务指标（最实际）。

面试加分："我更看重**稳定性**——k 稍微变化时簇结构是否稳定；如果 k=4 和 k=5 的划分看起来完全不同，说明数据本身没有清晰的簇结构。"

B33 ★★ KMeans 的复杂度？怎么加速？

A：**O(n · k · d · t)**（n 样本、k 簇、d 维、t 迭代轮数）。空间 O(n · d + k · d)。

加速手段： 1. **MiniBatch KMeans**：每步只用一个小 batch 更新中心（`O(b · k · d · t)`），大规模数据首选。 2. **Elkan 算法**：用三角不等式剪枝，跳过不可能更近的中心计算。 3. **kd-tree / ball-tree**：低维（d < 20）时加速最近邻搜索。 4. **降维**（PCA/随机投影）后再聚类。 5. **初始化用 k-means++ + n_init 小**（减少迭代次数）。

B34 ★★ KMeans 的假设与失效场景？

A：四个隐含假设：

1. **簇是凸的、球形的**（欧氏距离隐含等向性）→ 遇到月牙形/环形数据直接失效（该用 DBSCAN/谱聚类）。
2. **各簇方差相近、大小相近** → 一个大簇会被劈成两半去迁就小簇。
3. **欧氏距离有意义** → 高维下距离集中（所有点距离都差不多），聚类失效。
4. **对异常值敏感** → 均值被拉走（可用 k-medoids / 截断损失）。

另外：它只给**硬分配**，不能表达"某个点 60% 属于簇 A"（要软分配用 GMM）。

B35 ★★ KMeans / GMM / DBSCAN / 层次 / 谱聚类怎么选？

A：

方法	核心思想	优点	缺点
KMeans	中心 + 硬分配	快、简单、可扩展	球形假设、要指定 k、异常敏感
GMM（EM）	高斯混合 + 软分配	概率输出、允许不同协方差（椭圆簇）	要指定 k、对初始化敏感、高斯假设
DBSCAN	密度可达	不用指定 k、能找任意形状、能识别噪声	对 eps/minPts 敏感、密度差异大时失效、高维差
层次聚类	自底向上合并 / 自顶向下分裂	不用指定 k、给树状结构、可解释	$O(n^2)$/$O(n^3)$、不可回溯（贪心合并）
谱聚类	图拉普拉斯特征分解	能处理非凸簇	要指定 k、大图贵（需稀疏近似）

KMeans 与 GMM 的关系（高频追问）：KMeans 是 GMM 的**特例**——所有簇共享各向同性协方差 $\sigma^2 I$ 且 $\sigma \to 0$ 时的硬分配极限。EM 的 E 步是软分配（后验概率），M 步更新均值/协方差/权重。

B36 ★ 空簇和初始化有哪些实际陷阱?

A：三个：

1. **空簇**：某轮没有点分给它 → 均值无定义（除零）。处理：保留旧中心、或把它重置到"离其他中心最远的点"、或直接删掉该簇（k 减一）。
2. **初始中心重复**：随机抽到同一个点（k 接近 n 时概率不低）→ 退化。用 `replace=False` 抽样或 k-means++。
3. **用整数存中心**：均值可能是小数，用 int 会累积误差甚至震荡不收敛（这是实现 bug，但很常见）。

收敛判据的选择：用"中心移动量 < tol"而不是"标签不再变化"——后者在某些数据上会出现**标签循环震荡**（目标函数不变但分配在两组等价解之间跳）。

B5 自注意力（B37–B45）

B37 ★★★ 写出 self-attention 的完整定义，并说明每一步的形状。

A：

```
Q = X · W_Q,  K = X · W_K,  V = X · W_V          X: [L, d],  W: [d, d]
A = softmax( Q · Kᵀ / √d_k + mask , dim=-1)  Q · Kᵀ: [L, L]
O = A · V                                   [L,L]×[L,d] → [L, d]
```

三步的语义： 1. **打分**：`Q · Kᵀ` 得到任意两 token 的相似度（**全局**、与距离无关）； 2. **归一化**：逐行 softmax → 每个 query 对所有 key 的注意力分布（和为 1）； 3. **聚合**：用注意力权重对 V 加权求和 → 每个位置得到"按相关性混合"的新表示。

三个矩阵的不同角色：Q 是"我要找什么"，K 是"我能被什么匹配"，V 是"匹配上之后提供什么内容"。

B38 ★★★ 为什么除以 √d_k? 给出方差论证。

A：设 `q, k` 各维独立同分布、均值 0、方差 1，则点积：

```
q · k = Σ_{i=1}^{d_k} q_i · k_i
E[q · k] = Σ E[q_i]E[k_i] = 0
Var[q · k] = Σ Var(q_i k_i) = Σ E[q_i²] · E[k_i²] = d_k        (独立性 + 零均值)
```

所以点积的**标准差是 √d_k**——d_k=64 时量级约 8，d_k=256 时约 16。softmax 的输入尺度一大，就会进入**饱和区**（最大项 ≈1、其余≈0），此时雅可比 `diag(p)−ppᵀ ≈ 0`，**梯度消失**。

除以 √d_k 后方差回到 1，softmax 落在梯度良好的区间。**这也是 d_k 越大越需要缩放的原因。**

B39 ★★★ self-attention 的复杂度? 优化方向有哪些?

A：

```
时间: O(L² · d)      (打分 L²d + 加权 L²d)
显存: O(L² + L · d)  (要存注意力矩阵 L²)
```

L 是序列长度，所以**长上下文是平方级代价**。五个优化方向：

1. **IO 优化**：**FlashAttention**——分块（tiling）+ 在线 softmax，不把 L×L 矩阵落到显存，**省显存和 HBM 读写，不减少 FLOPs**。
2. **稀疏/滑窗**：Longformer（滑窗 + 全局 token）、BigBird（随机 + 窗口 + 全局）。
3. **低秩/核近似**：Linformer（对 K/V 降维）、Performer（核函数近似 softmax）。
4. **共享 KV**：MQA/GQA（主要省推理 KV cache）。
5. **换范式**：线性注意力、状态空间模型（Mamba，线性复杂度）。

B40 ★★★ 多头的作用是什么? 参数量怎么算?

A：把 `d` 维按**列**切成 h 份，每份 `d_k = d/h`，各自做 attention，再把 h 个 `[L, d_k]` 输出**按列拼回** `[L, d]`，最后过输出投影 `W_O`。

作用：不同头在**不同的子空间**学不同的依赖模式（相邻、句法、指代、跨模态对应），类似 CNN 的多通道分工。**计算量几乎不变**（$h \cdot L^2 \cdot (d/h) = L^2 \cdot d$），只是注意力的"视角"变多。

参数量：$4d^2$（Q/K/V/O 各 d×d）+ bias $4d$，**与 h 无关**（这是常被考的常识）。

B41 ★★★ 为什么需要位置编码？四种方案的取舍？

A：**self-attention 是置换等变的**（打乱输入只是打乱输出行），本身不含顺序信息，所以必须显式注入位置。

方案	做法	优点	缺点
正弦绝对编码	不同频率的 sin/cos	无需学习、可外推到更长	表达力弱、外推质量差
可学习绝对编码	每个位置一个向量	简单、效果好	**不能外推**（超过训练长度没有参数）
RoPE（旋转）	把 q/k 的相邻两维按位置旋转	编码**相对**位置、可外推、无额外参数	外推需插值/NTK 调整
ALiBi	注意力分数按距离加线性负 bias	外推好、实现极简、无参数	表达力略弱于 RoPE

RoPE 为什么能外推：因为 $q_m \cdot k_n$ 只依赖相对距离 $(m-n)$，所以理论上长度无关；但直接外推到远超训练的尺度会失真，所以实践要配位置内插（PI）、NTK-aware 缩放、YaRN。

多模态的扩展：图像/视频需要 2D/3D 位置（高、宽、时间），如 Qwen-VL 的 **M-RoPE**（把旋转维度分给 t/h/w）。

B42 ★★ attention 里的 mask 有哪几种？实现要注意什么？

A：三种：

1. **causal mask**（因果/自回归）：位置 i 只能看到 ⩽ i 的 key → 上三角置 $-\infty$。
2. **padding mask**：一个 batch 里不同的有效长度，key 侧的 pad 位置屏蔽。
3. **结构/跨模态 mask**：如视锥裁切（BEV query 看不到的相机）、跨样本隔离。

实现注意：- 用加性 mask（$scores + (-1e9 \text{ 或 } -inf)$）或 $masked_fill$，**统一语义**（True 是屏蔽还是保留，PyTorch 的 SDPA 里 bool mask 是 $True = 允许$，与 $masked_fill$ 相反）。- **一行全被屏蔽 → 分母为 0 → NaN**：要么保证每行至少一个有效位，要么对全屏蔽行单独处理。

B43 ★★ "置换等变"具体是什么意思？有什么后果？

A：设 P 是置换矩阵，则 $Attention(PX) = P \cdot Attention(X)$ ——**输入换顺序，输出只是跟着换顺序，内容不变**。

两个直接后果：1. **必须加位置编码**（否则模型分不清"猫追狗"和"狗追猫"）；2. **注意力的权重矩阵不含"距离"概念**（不像卷积的局部性）——所以它对"相邻"这件事没有先验，全靠学。

顺带一个面试题：为什么 Transformer 能表达卷积？→ 可以用特定的注意力模式（固定的相对位置偏置）模拟卷积核的行为，即"Transformer 的表达力包含卷积"。

B44 ★★ MQA / GQA / FlashAttention / 线性注意力 / Mamba 各解决什么？

A：

方法	解决什么	代价
MQA（Multi-Query）	所有头共享 1 组 K/V → KV cache ÷h	表达力略降
GQA（Grouped-Query）	h 个头分 g 组共享 K/V → ÷(h/g)	折中，主流选择
FlashAttention	分块 + 在线 softmax，不落 L^2 矩阵 → **省显存与 HBM 读写**	算法等价，不改 FLOPs
线性注意力（Performer/Linformer）	核近似/低秩 → **O(L) 复杂度**	近似误差、长程依赖略弱
Mamba / SSM	选择性状态空间 → 线性复杂度、推理 O(1) 每步	表达力待验证、生态不成熟

一句话："MQA/GQA 省的是**推理时的 KV cache**；FlashAttention 省的是**显存和带宽**；线性注意力/Mamba 改的是**复杂度阶**——三者解决的是不同层面的问题。"

B45 ★ self-attention 与 CNN / RNN 的本质区别?

A:

	CNN	RNN	Self-Attention
感受野	局部（靠堆叠扩大）	理论上全局（实际衰减）	**每层全局**
并行性	高	**低**（时序依赖）	高
归纳偏置	强（局部性、平移等变）	中（序列性）	**弱**（几乎无先验）
复杂度	O(L · k^2)（与长度线性）	O(L)（串行）	**O(L^2)**
长依赖	需深层	梯度消失/爆炸	直接建模（但可能"注意力分散"）

取舍结论："**归纳偏置越强，小数据越好、大数据越受限**。CNN 在数据少时更强，Transformer 在数据够时上限更高——这也是 ViT 需要大数据/强增广/蒸馏才超过 CNN 的原因。"

附：复习优先级

优先级	题目
必须能讲（A 部分 12 题）	A1 A2 A3 A7 A13 A14 A23 A24 A31 A32 A33 A39
必须能推（B 部分 14 题）	B2 B3 B9 B12 B13 B14 B17 B18 B20 B29 B30 B37 B38 B40
能讲 1 分钟（其余 A）	A4–A6 A8–A12 A15–A22 A25–A30 A34–A38 A40–A45
能讲 1 分钟（其余 B）	B1 B4–B8 B10 B11 B15 B16 B19 B21–B28 B31–B36 B39 B41–B45

B 部分最容易被追问的三个推导：`softmax+CE 的梯度 p−y`（B3）、`为什么除以 √d_k`（B38）、`KMeans 为什么局部最优`（B30）。这三个能白板推出来，ML 基础这一关基本不会挂。